Introduction & Setup

PSY 410: Data Science for Psychology

Dr. Sara Weston

2026-03-30

Why are we here?

Psychology has a data problem

In 2015, a team of 270 researchers tried to replicate 100 published psychology studies.

. . .

Only 36% produced the same results.

. . .

This wasn't fraud. These were real labs, following published methods, using real data.

So what went wrong?

Spend ~5 minutes on this opening sequence. Let the "36%" land — pause after the reveal. This is the Open Science Collaboration (2015) study. Students may have heard of the replication crisis in intro psych but probably don't know the details. The goal is to frame data science skills as the *solution*, not just a technical requirement. Don't get into specific studies yet — the point is the systemic problem.

Many things went wrong

- Analyses that couldn't be reproduced (even by the original authors)
- Data that was cleaned by hand with no record of what changed
- Figures that obscured rather than revealed patterns
- Code that only worked on one person's laptop

. . .

The replication crisis isn't just a statistics problem. It's a **workflow** problem.

The "workflow problem" reframe is the key insight. Students expect this to be a stats problem — redirect that assumption. This sets up the entire course narrative: every skill we teach is about building a better workflow. Transition: "So what does a better workflow look like? That's what this course is about."

This course is about the workflow

Every technical skill we learn this quarter serves the same goal:

. . .

> Start with raw data. End with a clear, honest, reproducible story.

. . .

That means learning to:

- **Import** data without breaking it
- **Transform** it transparently (with code, not clicks)
- **Visualize** it to find patterns — and catch errors
- **Communicate** what you found so others can verify it

Read the blockquote aloud — it's the course mission statement. The four bullet points map directly to the four units of the course. Students don't need to memorize this now, but it gives them a roadmap. Spend ~2 minutes here.

About this course

What we'll learn:

- Data visualization with ggplot2
- Data transformation with dplyr
- Data tidying with tidyr
- Reproducible reports with Quarto

Why it matters:

- Psychology is increasingly data-driven
- These skills make your work reproducible
- They transfer to any career that touches data

How this course works

Your grade

Component	Weight	What it is
Assignments	35%	8 weekly coding exercises
Reading quizzes	15%	10 pre-class quizzes — unlimited retakes, best score counts
Participation	15%	In-class pair coding submissions
Final project	35%	Analyze a dataset of your choosing

Students fixate on grading — spend ~3 minutes walking through this clearly. Emphasize: quizzes are points-based but they can retake as many times as they'd like and you always take the best score, participation is about showing up and trying (not correctness), and the final project is the biggest piece. The 48-hour late policy applies to everything — mention it but don't belabor it now.

Weekly assignments

- Short, focused coding exercises that reinforce each session's content
- Assigned on Wednesdays, due the following Sunday at 11:59 PM
- Designed to take 1–2 hours outside of class
- You'll get time during class to start and ask questions

. . .

8 assignments total. Late submissions accepted up to 48 hours with a 10% daily penalty.

Reading quizzes

Each week, a brief Canvas quiz on that week's readings. Due **Sunday at 11:59 PM**.

. . .

Each quiz pulls **5 random questions** from a larger bank. You can retake as many times as you want — **your best score counts.**

. . .

10 quizzes total (one per week). Same late policy as assignments.

In-class participation

Every class, you'll work with a partner on a coding exercise.

- Submit your attempt on Canvas before you leave
- Graded on completion and good-faith effort, not correctness
- This is your time to experiment, make mistakes, and learn

The final project

A capstone project where you apply everything to a dataset you choose.

Milestone	When
Proposal	Week 5 — dataset and research questions
Draft	Week 8 — working code and preliminary results
Final report	Week 10 — polished Quarto document
Presentation	Finals week — 5-minute share-out

The team challenge

The team challenge should feel fun, not stressful. Emphasize that it doesn't affect grades — it's just a way to build community and motivate each other. Teams will be formed from the survey they take today, balanced by prior experience. The winning team gets a pizza celebration on the last day. Spend ~3 minutes on both team challenge slides.

At the start of the term, you'll be placed on a team of 5–6 students.

Throughout the quarter, your team earns points:

- **Pair coding** — team gets a point when (nearly) everyone submits
- **Assignments** — team gets a point when everyone submits on time
- **Quizzes** — team with the highest average earns a point
- **Fun challenges** — weekly mini-challenges outside of class

The team challenge

The team challenge **does not affect your grade.**

The winning team at the end of the term earns a celebration.

. . .

We'll form teams based on a short survey you'll take today. Teams will be announced on Wednesday.

The tools we'll use

The old way vs. the new way

Point-and-click (SPSS, Excel)

- Hard to reproduce
- Error-prone
- Limited visualizations
- Doesn't scale

Code-based (R)

- Fully reproducible
- Transparent & shareable
- Unlimited customization
- Handles any data size

Why R specifically?

What you need	R delivers
Free	Open source — no licenses, ever
Built for data	Created by statisticians, not software engineers
Psychology packages	`psych`, `lavaan`, `lme4`, `brms`
Publication-quality figures	`ggplot2` — industry-leading
Reproducibility	Quarto integration (we'll learn this)

Students often ask "Why not Python?" Honest answer: both are great, but R's ecosystem is purpose-built for statistical analysis and psychology has strong R adoption. The packages listed here (`lavaan` for SEM, `lme4` for mixed models) are what they'll encounter in grad school or advanced stats courses. Don't spend more than ~2 minutes here — they'll see the proof through the course.

Getting set up

What you need to install

1. **R** — the programming language
   - Download from cloud.r-project.org
2. **RStudio** — the interface we'll use to write R
   - Download from posit.co/download/rstudio-desktop

. . .

> **!** Important
>
> Install R **first**, then RStudio. RStudio needs R to work!

Students should have installed R and RStudio before class (it was in the pre-class instructions), but expect ~20% to not have done it. Have them start installing now while you continue — installations take a few minutes. Common issues: Mac users need to choose the correct chip (Apple Silicon vs Intel); Windows users may need admin permissions. If someone is truly stuck, they can pair with a neighbor for today.

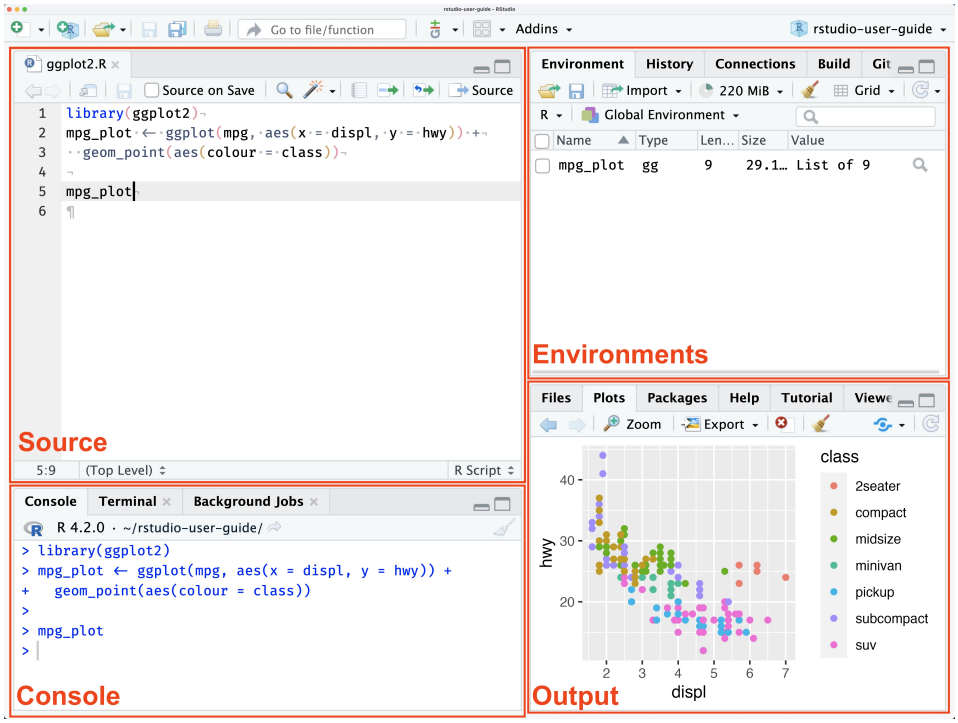
R vs RStudio

Think of it like this:

- **R** is the engine of a car
- **RStudio** is the dashboard, steering wheel, and GPS

You *could* drive with just an engine… but why would you?

The RStudio interface



The four panes: Source (top-left), Console (bottom-left), Environment (top-right), Files/Plots/Help (bottom-right). Walk through each pane briefly — students will get to know them organically, so don't over-explain. The most important thing right now is that the Console is where code runs and Source is where you save code. The Environment pane will make more sense once they create objects.

The four panes

Pane	What it does
Source (top-left)	Write and edit your code files
Console (bottom-left)	Run code interactively, see output
Environment (top-right)	See your data and objects
Files/Plots/Help (bottom-right)	Navigate files, view plots, get help

Let's try it: The Console

Type in the Console and press Enter:

```r
2 + 2
```

. . .

```r
2 + 2
```

```
[1] 4
```

Congrats — you just ran R code!

This is the first interactive moment — have everyone type `2 + 2` in their Console and press Enter. Wait until most people have done it. This tiny success matters: it proves R is working and they can make it do something. If anyone gets an error here, it's usually a sign R didn't install correctly.

The assignment operator

In R, we use `<-` to assign values to objects:

```r
x <- 10
```

Think of it as an arrow pointing left: "put 10 into x"

. . .

```r
# You can use objects in calculations
x * 2
x + x
```

. . .

> Tip
>
> **Keyboard shortcut:** Alt + - (Windows) or Option + - (Mac)

The assignment operator is a common early confusion. Students will try to use `=` (which works but isn't idiomatic R). Don't fight this battle too hard — just model `<-` consistently and they'll adopt it. The keyboard shortcut is worth demonstrating live — have everyone try it. Also note that the object `x` now appears in the Environment pane (top-right).

Organizing your work

Two mental models

This filing cabinet vs. laundry basket metaphor (from Scott Cunningham's Gov 51 at Harvard) is one of the most important conceptual lessons in the course. Spend ~5 minutes on the two "mental models" slides combined. Most students have grown up with search-based file access (Spotlight, Google Drive) and have never thought about directory structure. The goal is to make them understand *why* organized folders matter for code — not just as a preference but as a functional requirement.

The filing cabinet

Figure 1: Filing cabinet with labeled drawers, representing an organized project file structure

The laundry basket

Figure 2: Overflowing laundry basket, representing files scattered without structure

Two mental models

The filing cabinet

Every piece of paper has a **place** where it *lives*.

You navigate **to** that place to find it.

```
Documents/
  psy410/
      data/
      scripts/
  thesis/
      data/
      drafts/
```

The laundry basket

Everything is in **one pile**.

You **search** to find it.

Google, Spotlight, Finder search, "Recent files"…

Why coding requires the filing cabinet

When you write:

```r
survey <- read.csv("data/raw/survey_responses.csv")
```

You're giving directions: "Start here. Go into `data`. Then into `raw`. Find `survey_responses.csv`."

. . .

If the file isn't exactly there, the code **breaks**. No fuzzy matching. No "did you mean…?"

. . .

> **Important**
>
> This is why we need to learn directory structure — even if it feels foreign.

This is the "aha" moment for the filing cabinet metaphor. When students get "file not found" errors later in the course, point them back to this concept. The code gives *directions*, not

a *search query* — if the path is wrong, R can't find the file, period. This is the single most common source of frustration for beginners.

Why use RStudio Projects?

Without projects:

- Files scattered everywhere
- `setwd()` nightmares
- "It works on my computer"
- Lost work when switching tasks

With projects:

- Everything in one folder
- Paths just work
- Share the whole folder
- Easy to switch between projects

. . .

> Tip
>
> **The golden rule:** Someone else should be able to run your code and get the same results. That "someone else" includes Future You — who has forgotten everything.

Creating a project

1. File → New Project…
2. Choose "New Directory" → "New Project"
3. Name it (e.g., "psy410")
4. Pick a location (Documents folder is fine)
5. Click "Create Project"

. . .

You'll see a `.Rproj` file appear — this is your project file. From now on, double-click it to open your project.

Do this live — walk through the steps while students follow along. Pause after each step. The most common mistake is students creating the project inside another project, or not being able to find where they saved it. Encourage them to put it somewhere easy to find (Desktop or Documents). After creating the project, show them that the Files pane now shows the project directory.

A standard project structure

```
psy410/
  psy410.Rproj
  data/
     raw/           # Original data (READ ONLY)
     clean/         # Processed data
  scripts/
     01_clean.R     # Data cleaning
     02_analyze.R   # Main analysis
     03_visualize.R # Figures
  output/
      figures/       # Saved plots
```

A standard project structure

Two principles:

1. **Raw data is sacred** — never modify original files
2. **Outputs are disposable** — you can always regenerate them from code

These two principles are worth repeating throughout the course. "Raw data is sacred" means never edit a CSV by hand — always use code to clean it, so there's a record. "Outputs are disposable" means if you lose a figure, you just re-run the code. This connects back to reproducibility: if your workflow is code-based, everything is recoverable.

Naming things

Good names make code **self-documenting**. Bad names create confusion and bugs.

Do this:

- `reaction_time`
- `mean_anxiety`
- `survey_clean.csv`
- `01_clean_data.R`

Not this:

- `x1`, `temp`, `foo`
- `AvgAnx` (hard to read)
- `data_final_v2_REAL.csv`
- `stuff.R`, `untitled3.R`

The test: Can someone unfamiliar with your project understand what a variable contains or what a file does?

Get a quick laugh from the "data_final_v2_REAL.csv" example — everyone has done this. The naming slide is quick (~1 minute) but plants a seed. You'll reinforce good naming habits throughout the course, especially in the code style section of Session 4.

R Scripts

R scripts (`.R` files) are where you write and save your code.

To create one:

1. File → New File → R Script
2. Or press Ctrl/Cmd + Shift + N

. . .

Always save your scripts! The console history disappears.

Running code from a script

- **Run one line:** Put cursor on line, press Ctrl/Cmd + Enter
- **Run selection:** Highlight code, press Ctrl/Cmd + Enter
- **Run entire script:** Ctrl/Cmd + Shift + Enter

. . .

> Tip
>
> You'll use Ctrl/Cmd + Enter constantly. Memorize it!

Comments

Use `#` to write comments — notes for yourself and others:

```
# This calculates the mean age of participants
mean_age <- mean(ages)

# Anything after # is ignored by R
x <- 10  # this assigns 10 to x
```

Comments are essential. Your future self will thank you.

Your first real task

Installing packages

R's power comes from **packages** — bundles of functions others have written.

```
# Install a package (do this once)
install.packages("tidyverse")
```

```
# Load a package (do this every session)
library(tidyverse)
```

The tidyverse

The `tidyverse` is actually a collection of packages:

Package	Purpose
`ggplot2`	Data visualization
`dplyr`	Data manipulation
`tidyr`	Data tidying
`readr`	Reading data files
`tibble`	Modern data frames
`stringr`	String manipulation
`forcats`	Working with factors
`purrr`	Functional programming

We'll use most of these throughout the course.

Don't go through every package — just note that `library(tidyverse)` loads them all at once. Students sometimes worry about memorizing all of these. Reassure them: we'll learn them one at a time, and they only need `library(tidyverse)` at the top of every script. We won't cover `purrr` in this course.

Let's look at some data

After loading tidyverse, you have access to built-in datasets:

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.1.5
v forcats   1.0.1     v stringr   1.5.2
v ggplot2   4.0.2     v tibble    3.3.1
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.1.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
# mpg is a dataset about fuel economy
mpg
```

```
# A tibble: 234 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 audi         a4           1.8  1999     4 auto~ f        18    29 p     comp~
 2 audi         a4           1.8  1999     4 manu~ f        21    29 p     comp~
 3 audi         a4           2    2008     4 manu~ f        20    31 p     comp~
 4 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
 5 audi         a4           2.8  1999     6 auto~ f        16    26 p     comp~
 6 audi         a4           2.8  1999     6 manu~ f        18    26 p     comp~
 7 audi         a4           3.1  2008     6 auto~ f        18    27 p     comp~
 8 audi         a4 quattro   1.8  1999     4 manu~ 4        18    26 p     comp~
 9 audi         a4 quattro   1.8  1999     4 auto~ 4        16    25 p     comp~
10 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
# i 224 more rows
```

Your turn!

1. Create a new RStudio Project called "psy410"
2. Inside it, create three folders: `data`, `scripts`, `output`
3. Create a new R script in `scripts/` and save it as `01_practice.R`
4. Install and load the tidyverse
5. Run `glimpse(mpg)` and `glimpse(diamonds)`
6. Save your script!

Give students ~10 minutes for this. Walk around the room and help. The most common issues: (1) students who haven't installed R/RStudio yet — have them start installing now; (2) students who don't know how to create folders — show them the "New Folder" button in

the Files pane; (3) `install.packages("tidyverse")` taking a long time — this is normal on first install, can take 3–5 minutes. If they finish early, encourage them to try other things: `nrow(mpg)`, `names(mpg)`, `head(mpg)`.

Getting help

When you're stuck

1. **Use `?`** — e.g., `?mean` opens the help page
2. **Google it** — seriously, everyone does this
3. **Stack Overflow** — most R questions are answered there
4. **R4DS book** — r4ds.hadley.nz
5. **Ask me** — that's what I'm here for!

Reading help pages

```
?mean
```

Help pages include:

- **Description** — what the function does
- **Usage** — how to call it
- **Arguments** — what inputs it takes
- **Examples** — working code you can run

The examples section is gold — run them!

Error messages

Errors will happen. A lot. That's normal.

```
# This will error - can you spot why?
maen(c(1, 2, 3))
```

```
Error in maen(c(1, 2, 3)): could not find function "maen"
```

. . .

Read the error message carefully — R is trying to help you.

The typo example (`maen` instead of `mean`) is intentional — it shows the most common beginner error. Normalize errors loudly: "I make typos every single day. R will yell at you. That's fine." This sets the emotional tone for the whole course. Students who fear errors will freeze up; students who expect errors will debug and learn.

Why no AI? (Yet)

The bicycle vs. the motorcycle

Steve Jobs called the computer a **"bicycle for the mind"** — it amplifies your pedaling, but *you* provide the balance and direction.

Generative AI is more like a **motorcycle**. It provides the engine. But if you don't know how to ride, you crash faster and harder.

Read Cat Hicks: "Cognitive helmets for AI bicycles"

The AI policy section should take ~5 minutes across the three slides. Anticipate pushback: some students will think banning AI is old-fashioned. The bicycle/motorcycle metaphor reframes it — we're not anti-AI, we're pro-fundamentals. The course AI policy (in the syllabus) allows AI for debugging help later in the term but not for generating assignment code. The key message: "I want you to be the pilot, not the passenger. We'll get to the motorcycle — but first, learn to balance."

The metacognition trap

AI gives you a working answer immediately. It *feels* like you solved it.

But you didn't build the **mental model** of *why* it works.

When the AI hallucinates (which it will), **you cannot debug it.** You are stranded.

Using AI now = skipping the gym but expecting to get strong.

Building the helmet first

Before we get on the motorcycle, we need a **cognitive helmet**:

1. **Syntax literacy** — reading code as fluently as English
2. **Debugging resilience** — emotional regulation when things break
3. **Strategic friction** — breaking big problems into small pieces

That's what the many short assignments in this course build. The goal: **you** are the pilot, not the passenger.

Coding teaches you to think differently

Learning R isn't just about R. It's about a set of habits that transfer everywhere:

. . .

- **Breaking big problems into small steps** — instead of staring at a question, you ask: *what's the first concrete thing I can do?*
- **Thinking in transformations** — data isn't a fixed object; it's something you act on, step by step
- **Debugging as a skill** — forming a hypothesis, testing it, revising — this is just the scientific method

. . .

You may never write another line of R after June. **That's okay.** The thinking you build here will show up everywhere: in a lab, in a dissertation, in any job that asks you to solve a messy problem.

This slide is a morale investment — plant it early and reference it when students get frustrated later in the term. The core message: the struggle is the point. Decomposing problems, testing hypotheses, tolerating ambiguity — these are transferable cognitive habits, not just coding tricks. Psychology students in particular often don't see themselves as "technical people." This reframes the course as building a way of thinking, not just a skill set. Keep this short (~1–2 minutes) and deliver it warmly — it should feel like reassurance, not a lecture.

The interview no one warns you about

Many data science and research jobs include a **technical interview** — you're given a dataset and asked to write code **live**, on the spot.

No AI. No Stack Overflow. No notes. Just you and a blank script.

You don't need to be ready for that by June. But every time you practice writing code from memory — instead of copying or prompting — you're building the confidence and fluency that will matter when it counts.

Start now, so you're not starting from zero later.

This slide lands well after the cognitive helmet framing — it gives students a concrete, career-relevant reason to practice without AI. Don't oversell the stakes ("you'll never get a job!") — the tone should be encouraging. The message: you're building a skill that compounds. Even if they're not headed for a data science career, being able to sit down and code without a crutch is a transferable confidence. Mention that live coding interviews are common in tech, research labs, and increasingly in social science positions that involve data work.

Wrapping up

Before next class

Read:

- R4DS Ch 2: Workflow basics
- R4DS Ch 6: Workflow: scripts and projects

Do:

- Complete the **team formation survey** on Canvas (if you haven't already)
- Install R and RStudio
- Create your psy410 project with `data/`, `scripts/`, and `output/` folders
- Install the tidyverse package
- Poke around! Try things! Break stuff!

Key takeaways

1. **Psychology needs better data practices** — that's what this course builds
2. **R is a tool** — like learning any tool, it takes practice
3. **Organization matters** — filing cabinet, not laundry basket
4. **Name things clearly** — your future self will thank you
5. **Errors are learning opportunities** — read them, Google them

The one thing to remember

Every skill you learn in this course is a brick in a wall between you and the replication crisis.

See you Wednesday for your first visualization!

End on this line — it ties back to the opening and gives them a sense of purpose. Before dismissing, remind them of the to-do list on the previous slide: readings, team survey, install R/RStudio, create project, install tidyverse. The team survey is the most time-sensitive — teams need to be formed before Session 2.